

pire2pire.com

Security Strategy

Presented by

Axel Houairi



SOMMAIRE

SOMMAIRE.....	2
INTRODUCTION.....	4
LA SÉCURITÉ EN GÉNÉRALE.....	5
1. Réglementation (RGPD).....	5
1.1. Transparence.....	5
1.2. Consentement.....	6
1.3. Sécurisation.....	6
1.4. Droits des Utilisateurs.....	6
2. Menaces et attaques.....	7
STRATÉGIE DE SÉCURISATION.....	10
1. Sécurisation de l'API.....	11
1.1. Communications.....	11
TLS.....	11
HSTS.....	12
CORS.....	12
DDOS.....	13
1.2. Authentification.....	14
Rôles.....	14
Identifiants.....	14
Email.....	15
Mot de passe.....	15
Jeton de session.....	16
1.3. Protection contre les attaques XXE (XML External Entity).....	16
2. Sécurisation de la base de données.....	17
2.1. Validation des données stockées.....	17
2.2. RBAC.....	18
2.3. Données sensibles.....	19
Mot de passe.....	19
Hachage avec SHA256.....	19

Salage.....	19
Identifiant (ID).....	20
2.4. Requêtes préparée.....	21
3. Sécurisation du front-end.....	22
3.1. TLS.....	22
3.2. Content Security Policy (CSP).....	23
3.3. Authentification.....	24
Inscription et connexion.....	24
Stockage local.....	24
SURVEILLANCES.....	26
1. Renforcement de la journalisation et des audits (Back-end, Base de données, Front-end).....	26
1.1. Audit de sécurité régulier.....	26
1.2. Journalisation et détection des anomalies.....	27
Principes de journalisation.....	27
Événements à journaliser.....	27
2. Gestion des dépendances et sécurisation du code (Back-end, Base de données, Front-end).....	28
2.1. Vérification des dépendances.....	28
2.2. Sécurisation de l'exécution des scripts.....	29
Restriction des fonctions dangereuses.....	29
Sandboxing des scripts.....	29
CONCLUSION.....	31

INTRODUCTION

pire2pire.com is a web application that offers various learning programs, connecting learners with coaches to guide them throughout their learning journey.

The platform facilitates interactive and personalized learning experiences by linking users who want to acquire new skills with experts in different fields.

In this context, data security and user privacy are critical challenges. The application processes sensitive information, including user identification data, learning progress, and communications between coaches and learners.

Any security breach could not only compromise this data but also damage the platform's reputation and undermine user trust.

To ensure a seamless user experience while maintaining compliance with the [RGPD](#), this document contains pieces of information to elaborate a security strategy designed to safeguard user data without compromising the accessibility and efficiency of the platform.

LA SÉCURITÉ EN GÉNÉRALE

1. Réglementation ([RGPD](#))

Certaines règles sont imposées par l'Union Européenne en termes de sécurité et de la collecte de données dans les sites internet.

L'application **pire2pire** devra s'y prêter et respecter les règles imposées par la [RGPD](#).

La plateforme **pire2pire** devra collecter les données personnelles de ses utilisateurs tel que leurs adresses mail, prénom, nom et informations de paiements.

1.1. Transparence

- Informer clairement les utilisateurs des données collectées, de leur finalité et de leur durée de conservation.
- Fournir une politique de confidentialité détaillée.
- Mentionner les bases légales justifiant la collecte.

1.2. Consentement

- Obtenir un consentement explicite avant de collecter des données personnelles (cases à cocher pour l'acceptation des conditions d'utilisation).
- Permettre aux utilisateurs de retirer leur consentement.

1.3. Sécurisation

- Mettre en place un système pour sécuriser toutes les couches de l'application pour éviter les fuites de données.

1.4. Droits des Utilisateurs

- **Droit d'accès** (savoir quelles données sont collectées)
- **Droit de rectification** (modifier des données incorrectes)
- **Droit à l'effacement** (supprimer leurs données)
- **Droit à la portabilité** (récupérer leurs données)
- **Droit d'opposition** (refuser certains traitements)

2. Menaces et attaques

L'application **pire2pire** comme tout site web est susceptible à certaines menaces et attaques, avant d'élaborer une stratégie il est important de prendre conscience de ces menaces et comprendre leur réelle but pour mieux s'en prévenir :

- **Le vol de données** est une attaque visant à compromettre la confidentialité des informations sensibles, telles que les identifiants, les informations personnelles ou bancaires.
- **La compromission des ressources** se produit lorsque l'intégrité du contenu d'un site web est violée, ce qui peut entraîner sa défiguration. Ce type d'attaque vise à altérer le contenu original du site pour y substituer un contenu frauduleux par l'attaquant.
- **Le déni de service** ou **DDOS** a pour but de rendre un site web inaccessible, soit en l'arrêtant complètement, soit en le ralentissant de manière significative.

Voici une liste d'attaques qu'il faut particulièrement prendre en compte et faire tout ce qui est nécessaire pour minimiser les risques que ces menaces représentent pour l'application **pire2pire**.

Cross-Site Scripting (XSS) :

- Injection de code (**JavaScript**) malveillant dans un site web pour piéger un utilisateur à effectuer des actions à son insu.

Cross-Site Request Forgery (CSRF) :

- Forcer un utilisateur à envoyer une requête sans son consentement souvent sur un site où il est déjà connecté à objectif de modifier ses informations ou même voler de l'argent.

Server-Side Request Forgery (SSRF) :

- Forcer un serveur à faire des requêtes vers d'autres ressources pour avoir accès aux ressources internes et récupérer des informations sensibles.

SQL Injection (SQLi) :

- Injecter du code **SQL** malveillant pour manipuler une **base de données** pour soit récupérer des informations ou même supprimer des données.

Local / Remote File Inclusion (LFI / RFI) :

- Charger et exécuter des fichiers non autorisés sur un serveur tel que lire des fichiers sensibles ou exécuter des scripts malveillants.

XML External Entity (XXE) :

- Exploiter le traitement d'un fichier XML pour accéder à des fichiers ou exécuter des requêtes réseau pour objectif de voler des fichiers sensibles, attaques SSRF et l'exfiltration de données.

STRATÉGIE DE SÉCURISATION

Avant d'expliquer ma stratégie en détail il est important de comprendre les fondamentaux en termes de sécurisation que je vais utiliser.

Défense en profondeur :

- Mettre en place plusieurs couches de protection pour réduire les risques. L'objectif est qu'en cas de faille dans une couche, les autres puissent toujours protéger l'application.

Réduction de la surface d'attaque :

- Réduire les points d'entrée qu'un attaquant pourrait exploiter. Moins il y a de portes ouvertes, plus l'application est difficile à attaquer.

Role-Based Access Control (RBAC) :

- Le RBAC est une méthode de gestion des droits d'accès qui attribue des permissions aux utilisateurs en fonction de leur rôle dans l'application, attribuant chaque rôle les permissions strictement nécessaires.

1. Sécurisation de l'API

1.1. Communications

TLS

La sécurisation via **TLS** est cruciale pour sécuriser les communications entre l'**API** et le **front-end**, elle transforme le protocole **HTTP** en **HTTPS**, elle empêche les attaques de type **Man in the middle**.

L'authenticité des communications

- Certificats permettant de vérifier l'authenticité des communications assurant ainsi que les utilisateurs communiquent bien avec l'**API** **pire2pire**.

Confidentialité des données

- Chiffre les données échangées entre le **front-end** et l'**API** pour protéger les informations des utilisateurs.

Intégrité des données

- Vérifie que les données échangées n'ont pas été altérées pour s'assurer que les informations reçues sont exactement celles qui ont été envoyées.

HSTS

HSTS est une sécurité qui force les navigateurs à utiliser **HTTPS** pour toutes les communications, pour l'application **pire2pire** nous allons mettre cette pratique en œuvre.

En-tête HSTS

- Utilisation de l'en-tête *Strict-Transport-Security* dans les réponses HTTPS via **helmet**.

Surveillance des Certificats Transparency Logs

- Utiliser des outils pour surveiller les CT logs et détecter les certificats illégitimes émis pour le domaine **pire2pire**, si un certificat illégitime est détecté il sera révoqué immédiatement.

CORS

L'utilisation de **CORS**(Cross-Origin Resource Sharing) est indispensable pour pouvoir contourner la **SOP**(Same-Origin-Policy) mise en place par la plupart des navigateurs pour pouvoir communiquer avec le **front-end** en y insérant le nom de domaine **pire2pire** pour remédier aux attaques **SSRF**.

CORS permet aussi de se protéger des attaques **CSRF** en définissant des règles précises sur les requêtes acceptées.

DDOS

Les attaques par déni de service distribué (**DDOS**) visent à submerger un serveur ou une **API** avec un grand nombre de requêtes, entraînant ainsi une surcharge et une indisponibilité du service.

Pour prévenir ces attaques, il est essentiel de mettre en place des mécanismes de limitation du trafic.

Une solution efficace consiste à implémenter un système de limitation de requêtes afin de restreindre le nombre de requêtes qu'un utilisateur peut effectuer sur une période donnée.

Dans notre cas, nous allons utiliser la bibliothèque **express-rate-limit**, qui permet de limiter les appels à l'**API** à 100 requêtes toutes les 10 minutes par adresse IP.

Cette approche permet :

- Réduire la charge sur le serveur
- Empêcher les utilisateurs malveillants de surcharger l'**API**
- Garantir un accès équitable aux ressources pour tous les utilisateurs

1.2. Authentification

L'authentification sera basé sur le principe de moindre privilèges avec des rôles attribués pour chaque type d'utilisateur afin de limiter les risques de permissions.

Rôles

Voici les rôles que nous allons mettre en place afin de catégoriser chaque utilisateurs de l'application :

- Super administrateur (tous les droits)
- Administrateur (droits de gérer les formateurs, apprentis, visiteurs et le contenu dans l'application)
- Formateur (droits de modifier ses apprenants)
- Apprenti (liée à une formation et un formateur)
- Visiteur (consultation des formations disponibles)

Identifiants

Les données stockées dans la **base de données** seront d'abord créées via l'**API**, il est donc important de sécuriser l'**API** en premier lieu puis sécuriser les systèmes de création de ses données tels que les identifiants, mot de passe, jetons de sessions et informations bancaires.

Email

L'utilisation de l'email comme identifiant pour l'authentification d'un utilisateur, plutôt qu'un simple nom d'utilisateur, présente plusieurs avantages en termes de sécurité et d'expérience utilisateur. Voici quelques raisons pour lesquelles nous allons procéder ainsi pour **pire2pire** :

- **L'email est unique** : Contrairement aux noms d'utilisateur, qui peuvent être choisis librement et parfois dupliqués ou usurpés, l'email est unique pour chaque utilisateur.
- **Difficile d'usurper** : il est plus difficile de deviner l'email d'une personne que de deviner un nom d'utilisateur courant.

Mot de passe

Les mot de passes seront sécurisés avec un système de **hachage** et **salage** avec **bcryptjs** pour ne pas stocker les mot de passes en clair mais en une suite de caractères illisibles(**SHA256**) dans la **base de données** afin d'empêcher les attaques de types **brute force**, **dictionnaire** ou d'intrusion dans la **base de données**.

Les mot de passe devront être composé de minimum 15 caractères contenant des chiffres et caractères spéciaux, cette obligation permettra d'avoir une sécurité élevée et moins de vulnérabilités, il sera fortement recommandé de changer de mot de passe tous les 1 à 3 ans.

Jeton de session

Les jetons(tokens) sont des éléments essentiels pour gérer l'authentification et l'autorisation dans une application web.

Les jetons de sessions pour **pire2pire** seront signés et chiffrés avec **JWT**(Json Web Token) et sécurisés via des jetons **CSRF** pour toutes les requêtes sensibles de types **POST**, **PUT** et **DELETE**.

Les jetons seront ensuite stockés dans des cookies et seront attribué une date d'expiration du jeton de 15 minutes, un système de **refresh token** sera aussi mis en place pour renforcer la sécurité des jetons et ne pas interférer la sessions des utilisateurs juste après que les 15 minutes soit écoulée.

Il y aura des restrictions pour l'envoi des cookies aux domaines de confiance en utilisant l'attribut *SameSite=strict*, un système de révocation sera également mis en place pour chaque jetons compromis afin de sécuriser les sessions des utilisateurs.

1.3. Protection contre les attaques XXE (XML External Entity)

Les attaques **XXE** exploitent les failles des parseurs **XML** mal configurés pour accéder à des fichiers sensibles ou exécuter des **requêtes malveillantes**. Afin de **protéger** l'application **pire2pire**, les entités externes seront désactivées.

2. Sécurisation de la base de données

2.1. Validation des données stockées

La validation des données est une étape cruciale dans la sécurisation de la **base de données**.

Avant de stocker des données dans une base, il est essentiel de s'assurer que celles-ci respectent certaines règles de format, de type, de longueur et d'intégrité.

Cela garantit non seulement la cohérence des données, mais également leur sécurité.

Dans le contexte de la validation des données, la bibliothèque **zod** joue un rôle central. Elle permet de définir des schémas de validation pour garantir que les données reçues correspondent aux attentes, tant sur le **front-end** que sur le **back-end**. Voici les raisons principales pour lesquelles nous allons utiliser cette méthode :

- Prévenir les erreurs de données
- Renforcer les données
- Assurer l'intégrité des données reçus
- Réduire la surcharge sur le serveur et afficher des erreurs précises si les entrées ne sont pas bonnes dans un formulaire

2.2. RBAC

Il est essentiel d'appliquer le principe de moindre privilège pour restreindre les permissions accordées aux utilisateurs et aux processus afin de limiter les risques d'accès non autorisé. Cela implique :

- De définir des rôles avec des accès spécifiques (lecture, écriture, etc).
- De restreindre les permissions des applications sur le système de fichiers.

Il y aura 5 types de rôles dans l'application **pire2pire** tel que :

<i>Super Administrateur</i>	<i>Administrateur</i>
Gérer les utilisateurs Attribuer et modifier des rôles Modifier des données Configurer le système	Gérer les formateurs, apprentis et utilisateur lambda Modifier certaines données Gérer les formations Accéder à toutes les données sauf celle réservée aux Super Administrateur
<i>Formateur</i>	<i>Apprenti</i>
Créer, modifier et supprimer des cours Gérer leurs apprentis Accéder aux informations des apprentis Modifier ses données	Accéder aux cours Passer des évaluations Modifier ses données Discuter avec son formateur
	<i>Visiteur</i>
	Consulter les formations S'inscrire

2.3. Données sensibles

Mot de passe

Pour sécuriser les mots de passe dans la base de données, ont va adopter une approche solide qui implique plusieurs étapes clés.

Hachage avec SHA256

Les mots de passe seront d'abord transformés en une chaîne de caractères illisible grâce à un algorithme de **hachage**. Ce processus rend le mot de passe original irrécupérable. Par exemple, au lieu de stocker le mot de passe en clair, nous allons le transformer en une chaîne fixe et unique de caractères, comme un "empreinte" du mot de passe.

Salage

Pour rendre le hachage plus sûr, un **salage** sera appliqué au mot de passe avant de le hacher. Le **sel** est une valeur aléatoire unique, ajoutée au mot de passe avant qu'il soit haché. Cela garantit que même si deux utilisateurs ont le même mot de passe, leurs hachages seront différents. Par exemple, si deux utilisateurs ont le même mot de passe "password123", l'ajout d'un sel unique pour chacun d'eux donnera des hachages différents. Le sel sera stocké dans la **base de données** avec le **hachage**, ce qui permet de vérifier le mot de passe plus tard.

Identifiant (ID)

L'utilisation d'un identifiant unique pour chaque utilisateur dans une **base de données** est cruciale pour garantir une gestion efficace et sécurisée des données. Par défaut, les identifiants sont souvent des nombres entiers qui sont générés automatiquement par la base de données (par exemple, avec un auto-increment). Cependant, l'utilisation d'un identifiant numérique présente certaines failles en termes de sécurité, notamment la prévisibilité et la vulnérabilité aux attaques. C'est pourquoi nous allons adopter un système basé sur des **UUID** (Universally Unique Identifier), qui permet d'améliorer la sécurité et l'intégrité de l'application.

Un **UUID** est un identifiant unique de 128 bits, généralement représenté sous forme de chaîne de caractères hexadécimaux, voici quelques raisons qui nous ont amené à ce choix d'utiliser des **UUID** :

ID unique garantie :

- Un **UUID** est extrêmement difficile à reproduire, même sur plusieurs systèmes.

Sécurité améliorée :

- Un **UUID** est long et composé de chiffres et de lettres, ce qui le rend extrêmement difficile à prédire.

2.4. Requêtes préparée

Les **requêtes préparées** sont l'une des techniques les plus efficaces pour sécuriser une **base de données** contre les attaques d'injection **SQL**. L'injection **SQL** est une vulnérabilité très courante dans les applications web où un attaquant peut insérer ou manipuler des requêtes **SQL** malveillantes via des champs d'entrée comme des formulaires, des URL ou des cookies. Cela permettrait à l'attaquant de récupérer, modifier ou supprimer des données sensibles dans la base de données, ou même de prendre le contrôle total de l'application. Pour prévenir ces attaques, l'utilisation de **requêtes préparées** est essentielle.

Nous allons mettre un système de requêtes **SQL** préparée pour l'application **pire2pire**, Voici les raisons qui nous ont poussé à cette conclusion :

Protection contre les injections SQL

- Comme les données sont envoyées séparément de la requête SQL, un attaquant ne pourra pas modifier la structure de la requête en insérant du code SQL malveillant.

Performances et gestion des erreurs améliorées

- Les requêtes préparées peuvent améliorer les performances et offrent une gestion plus robuste des erreurs.

3. Sécurisation du front-end

3.1. TLS

Comme indiqué dans la sécurisation de l'**API** il est important d'utiliser des en-têtes **HTTPS** afin de renforcer la sécurité pour les communications entre les serveurs.

Ce que **TLS** apporte :

- **Chiffrement des données** : Identifiants de connexion, détails de paiement, etc.
- **Authenticité du serveur** : Réduction de phishing ou d'attaques de type spoofing.
- **Protection** : Protection contre les attaques de type Man in the middle et eavesdropping.
- **Cookies de sessions** : Les cookies contiennent le jeton de session essentiel pour garder les utilisateurs connectés, TLS renforce la sécurisation de ces jetons et réduit les attaques de type hijacking.
- **Referencing (SEO)** : Meilleure référencement en général sur les navigateurs pour plus de visibilités du site.
- **Règlement** : Assurer une sécurité avec les en-têtes HTTPS sont essentiel pour respecter les réglementation imposées par la [RGPD](#)

3.2. Content Security Policy (CSP)

La **Content Security Policy** est un mécanisme de sécurité essentiel pour protéger les applications web contre diverses attaques, notamment les attaques de type **Cross-Site-Scripting** (XSS) et les injections de données.

Nous allons mettre en place **CSP** et **SRI** pour sécuriser l'application **pire2pire**, voici les points clés qu'apporte cette méthode :

Réduction des vulnérabilités XSS

- En limitant les sources de scripts autorisés, la **CSP** réduit le risque d'injection de scripts malveillants.

Protection contre le clickjacking

- La directive *frame-ancestors* empêche l'intégration de la page dans des iframes non autorisées

Contrôle des ressources externes

- La **CSP** permet de contrôler et de restreindre les ressources externes, réduisant ainsi le risque d'inclusion de contenu malveillant.

En mettant en œuvre ces pratiques et en configurant correctement la **CSP**, la sécurité de l'application **pire2pire** est renforcée et protège les utilisateurs contre diverses menaces.

3.3. Authentification

Inscription et connexion

L'**authentification** est un élément clé de la sécurité d'une application web, car elle permet de vérifier l'identité des utilisateurs.

Dans cette section, nous allons aborder l'importance de sécuriser le processus d'authentification côté **front-end**, en mettant l'accent sur la validation des données saisies dans les formulaires d'inscription et de connexion.

L'objectif est de garantir que les informations reçues sont conformes à nos attentes, de prévenir les attaques par **injection** et de garantir une expérience utilisateur fluide et sécurisée.

Stockage local

Lorsqu'un utilisateur se connecte à une application, il est essentiel de suivre son statut de connexion afin de lui offrir une expérience fluide sans qu'il ait besoin de se reconnecter à chaque action. Pour cela, nous utilisons des jetons de sessions, qui sont des informations cryptées permettant de maintenir la session active d'un utilisateur sur plusieurs requêtes.

Une fois l'utilisateur connecté, ces jetons sont stockés dans le navigateur afin d'être envoyés avec chaque requête ultérieure pour authentifier l'utilisateur.

Nous allons utiliser le **Storage** du navigateur pour stocker ce **cookie** contenant le jeton de session.

Voici les point clés de pourquoi nous allons utiliser cette méthode :

- **Accessibilité automatique** : Les cookies sont automatiquement envoyés avec chaque requête HTTP vers le domaine auquel ils sont associés, ce qui les rend parfaits pour gérer les sessions.
- **Contrôle de la durée de vie** : Les jetons de sessions auront une date d'expiration de 15 minutes, réduisant les risques d'usurpation.
- **Sécurité accrue** : Les cookies sont particulièrement utiles pour stocker des jetons de sessions car ils ont des attributs spéciaux permettant une sécurité renforcée.

Le stockage sécurisé des **jetons de session** est essentiel pour maintenir une expérience utilisateur fluide et sécurisée.

L'utilisation de **cookies** sécurisés avec les bons attributs (comme *httpOnly*, *Secure* et *Same-Site=Strict*) est une méthode fiable pour assurer que les jetons de session sont protégés contre les attaques courantes, telles que les attaques **XSS** et **CSRF**.

SURVEILLANCES

1. Renforcement de la journalisation et des audits (Back-end, Base de données, Front-end)

1.1. Audit de sécurité régulier

Pour garantir la sécurité continue de l'application, des **audits** réguliers seront réalisés afin d'**identifier** et de **corriger** les **vulnérabilités** potentielles dans le **back-end**, la **base de données** et le **front-end**. Ces audits peuvent inclure :

- **Tests d'intrusion** : Identifier les failles exploitables par un attaquant.
- **Audit du code source** : Vérification des bonnes pratiques de sécurité dans le back-end et le front-end.
- **Analyse statique et dynamique** : Détection des vulnérabilités à l'aide d'outils automatisés pour la base de données et le code applicatif.
- **Bug Bounty** : Implémentation d'un programme de signalement des failles par des experts externes.

1.2. Journalisation et détection des anomalies

Un système de **journalisation** efficace est essentiel pour **détecter** les activités suspectes et **analyser** les incidents de sécurité.

Principes de journalisation

- **Collecte centralisée des logs** : Mise en place d'un SIEM (Security Information and Event Management) pour analyser les événements de sécurité liés au back-end et à la base de données.
- **Horodatage des événements** : Synchronisation des horloges des serveurs pour garantir une traçabilité cohérente
- **Conservation des logs** : Stockage sécurisé des journaux d'événements avec une durée de rétention adaptée.

Événements à journaliser

- Tentatives de connexion échouées et réussies
- Modification critiques des permissions et rôles utilisateurs
- Actions sensibles comme les transactions financières ou modifications de données importantes

- Détection d'activités suspectes telles que les tentatives d'injection SQL ou XSS
- Anomalies détectées dans l'exécution des scripts côté front-end

2. Gestion des dépendances et sécurisation du code (Back-end, Base de données, Front-end)

2.1. Vérification des dépendances

L'utilisation de **bibliothèques** et de **frameworks** externes peut introduire des vulnérabilités si elles ne sont pas régulièrement **mises à jour**, que ce soit dans le **back-end**, la **base de données** et le **front-end**.

- **Utilisation de `npm audit` ou `yarn audit`** pour détecter les dépendances vulnérables dans le front-end et le back-end.
- **Mise à jour régulière des dépendances** avec `npm update` et `pip audit` tout en vérifiant la compatibilité.
- **Vérification des CVE (Common Vulnerabilities and Exposures)** pour les bibliothèques utilisées dans le code applicatif et les bases de données.

2.2. Sécurisation de l'exécution des scripts

Pour **minimiser** les risques liés à l'exécution de **scripts** malveillants, les pratiques suivantes doivent être appliquées sur le **back-end** et le **front-end** de l'application **pire2pire** :

Restriction des fonctions dangereuses

- Désactivation de l'usage de `eval()`, `setTimeout()` et `setInterval()` avec des chaînes de caractères dynamiques.
- Utilisation stricte de **CSP** (Content Security Policy) pour restreindre les sources autorisées de scripts côté **front-end**.
- Validation stricte des entrées utilisateurs pour éviter l'injection de code.

Sandboxing des scripts

- **Web Workers** : Séparation des tâches lourdes dans un environnement isolé en front-end.
- **Utilisation d'iframes avec l'attribut `sandbox`** pour exécuter du contenu tiers en limitant les permissions.

- **Définition des permissions minimales** pour les scripts tiers intégrés.

CONCLUSION

La sécurisation de l'application **pire2pire** repose sur une stratégie rigoureuse visant à protéger les données des utilisateurs tout en garantissant leur confidentialité, intégrité et disponibilité. En appliquant les principes de défense en profondeur, de réduction de la surface d'attaque et de contrôle d'accès basé sur les rôles (RBAC), nous avons défini des mesures robustes pour sécuriser l'**API**, la **base de données** et le **front-end**.

Toutefois, une bonne stratégie de cybersécurité ne peut être efficace sans un suivi et une surveillance continus. C'est pourquoi la journalisation et l'audit sont des éléments clés de notre approche. La journalisation permet de suivre en temps réel toutes les actions critiques au sein de l'application, telles que les tentatives de connexion, les modifications de données sensibles et les accès aux ressources protégées. En parallèle, l'audit régulier de ces journaux permet d'identifier toute anomalie, de détecter rapidement les incidents de sécurité et de garantir la conformité avec les réglementations en vigueur, notamment le [RGPD](#).

En intégrant ces mécanismes de surveillance et de contrôle, nous renforçons la capacité de **pire2pire** à réagir aux menaces et à maintenir un niveau de sécurité optimal pour tous ses utilisateurs.